Atividade 02

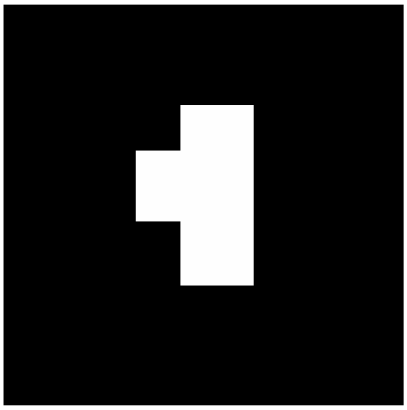
- **Aluno:** Manuel Ferreira Junior
- **Disciplina:** Processamento Digital de Imagens
- **Obs. (1):** Questões com necessidade de calculo de convolução, estarão todas as expressões no link, para facilitar a visualização e leitura da atividade. Além disso, cada questão em anexo possui uma copia da sua operação que deve ser realizada. Cada sheet do link em anexo, possui no seu titulo a questão referente e operação referente.
- **Obs. (2):** Com respeito as questões de implementação (8, 9 e 10), além do código disponibilizado no pdf, os links para os notebooks utilizados para as aplicações estão abaixo:
    1. `Questão 08:` 02_01_atv02_code_q08_sol1
    2. `Questão 09:` 03_atv02_code_q09
    3. `Questão 10:` 04_atv02_code_q10
    4. `[Rascunhos para validação de resoluções] Códigos para outras questões:` 01_atv02_code

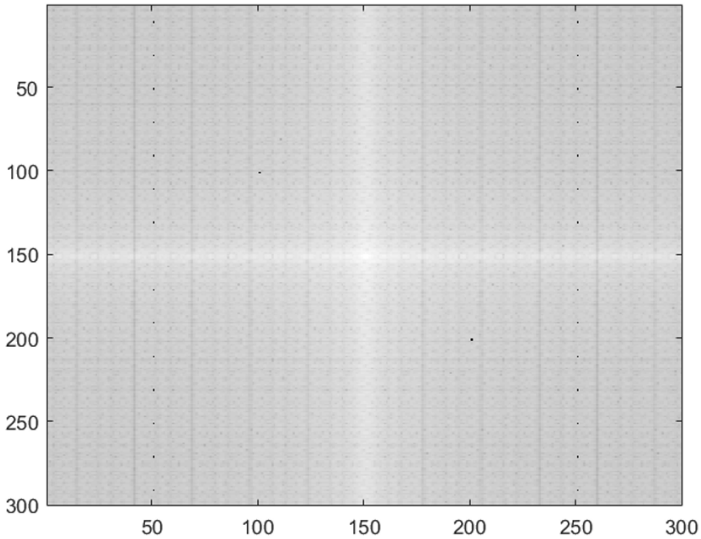
Imports

```
import math
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
from scipy.signal import convolve2d
from scipy.fft import ifft2
```

Questão 01

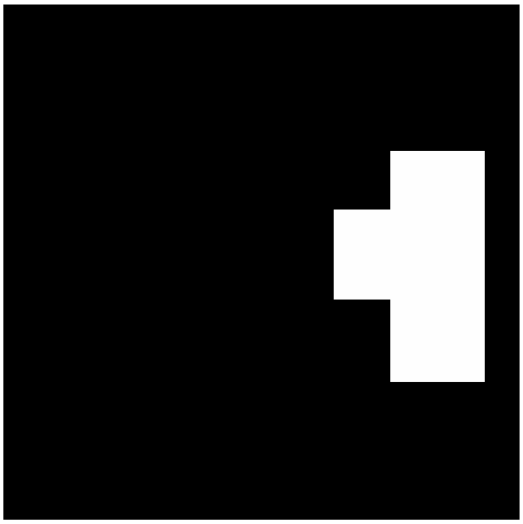
A imagem da esquerda tem a magnitude da sua transformada de Fourier apresentada à direita:

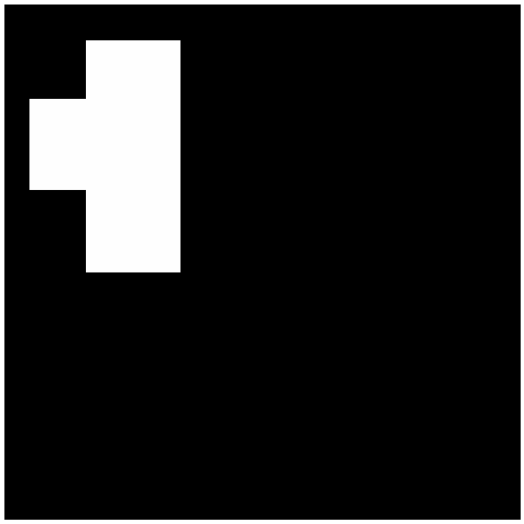




Como deve ser a mesma figura da magnitude da transformada de Fourier para as imagens abaixo? Considere que a única mudança, em relação à imagem apresentada acima e à esquerda, é o deslocamento do bloco branco. Justifique sua resposta.

Obs: Você não tem a imagem original para calcular a transformada. Apenas especule sobre o resultado esperado.





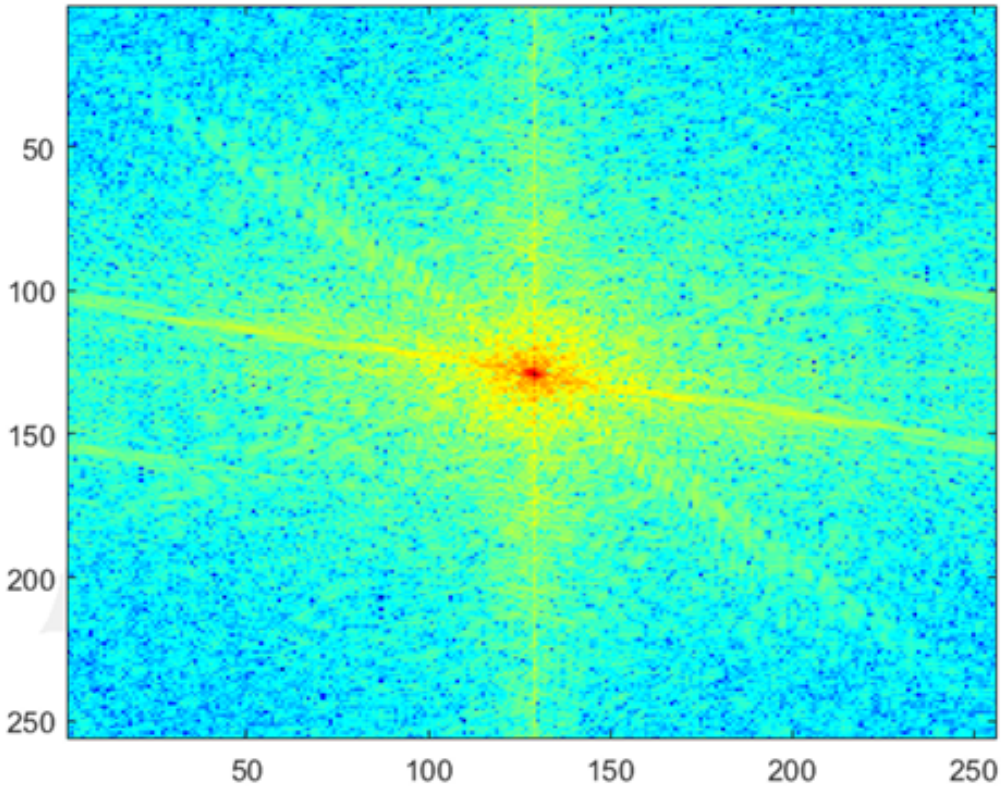
R.:

A Transformada de Fourier tem como foco principal analisar a distribuicao das frequências da iamgem e não elementos dispostos no espaço da imagem, ou seja, a posição de qualquer conteúdo na imagem caso seja alterado, de forma igual a não alterar a frequências presente na imagem, espera-se que o resultado da transformada de fourier permaneça igual. Logo, a transformada de fourier independe da localização do objeto na imagem, caso não exista variação de intensidade entre as imagens. Contudo, se esse objeto for rotacionado em alguma angulação, essa alteração pode ser expressa na transformada de fourier. Apesar da transformada ser igual, pois independe da localização do objeto na imagem e sim das frequências, a fase da transformada de fourier sera diferente, apesar de mesma magnitude.

Questão 02

Vimos, nos slides 16 e 17 da aula de filtragem, que o embaçamento de uma imagem provoca grandes mudanças na magnitude da transformada de Fourier dessa imagem. Considere a imagem abaixo:

Essa imagem gera a magnitude da transformada de Fourier a seguir:



Em comparação com a imagem da direita do slide 17, da aula de filtragem, vemos que o borramento parcial no canto inferior da direita, apesar de bastante forte, não está tão nítido na transformada. Sugira uma estratégia para detectar que houve esse distúrbio em alguma parte (bem definida) da imagem.

R.:

1) **Solução 01: Short Time Fourier Transform**

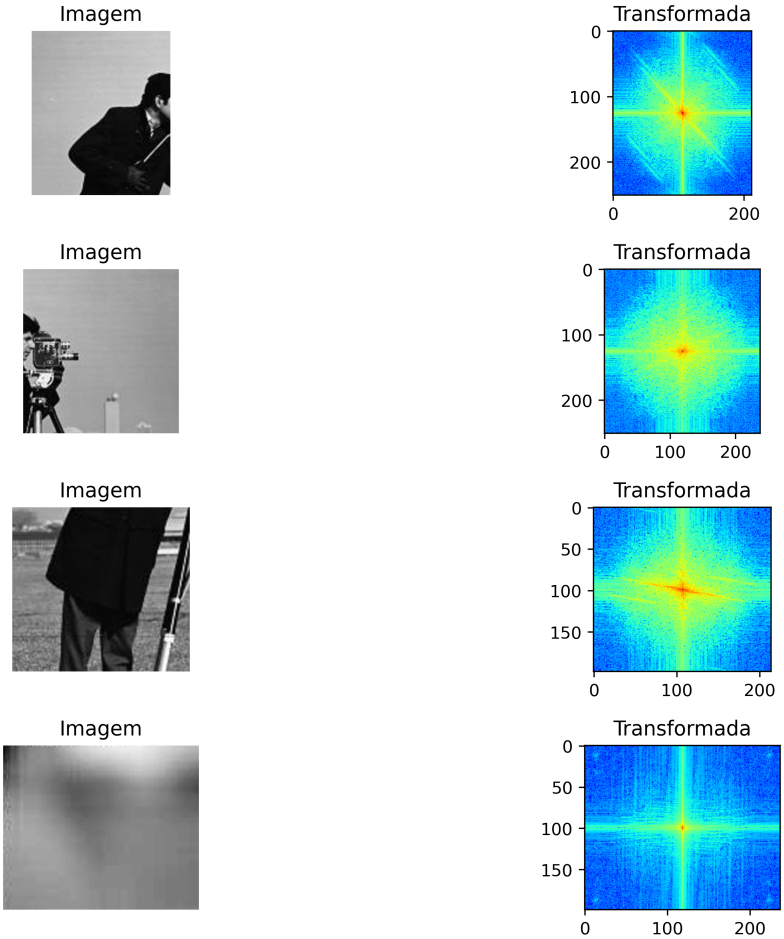
Uma técnica apresenta em sala que pode ser aplicada para a situação é a *Short Time Fourier Transform* (STFT) (Gabor, 1946), sendo uma técnica que utiliza uma janela deslizante para analisar regiões específicas de uma imagem de forma individual. Essa abordagem pode ser util para detectar borramentos localizados em quadrantes específicos da imagem.

Aplicando a STFT, o tamanho da janela e o salto possuem um impacto signficante na detecção de anomalias na imagem, sendo janelas menores e saltos mais precisos permitem uma análise mais detalhada, facilitando a identificação de regiões afetadas pelo borramento.

Essa técnica é eficaz para esse problema porque permite a análise local da imagem, dando enfase nas variações que poderiam passar despercebidas em uma análise global, sendo uma ferramenta poderosa para identificar problemas localizados em imagens, como borramentos em regiões especificas.

2) **Solução 02: Separar em quadrantes**

Uma variação da *STFT* mencionada anteriormente, pode ser a separação direta em quadrantes a imagem, reduzindo o campo de busca para ruidos, e podendo comparar as transformadas de fourier entre elas. Em especial no exemplo da questão, separar em quatro quadrantes pode ser conveniente, então aplicar a transformada de Fourier em cada quadrante. Pode-se entender esta solução como um *STFT* aonde as janelas são disjuntas mas complementares para a imagem original. A ideia é que, ao analisar o espectro individualmente para cada quadrante, será possível comparar as distribuições entre as regiões. O quadrante afetado pelo desfoque apresentará uma caracteristica diferente das outras, evidenciando o borramento. Na imagem abaixo, da para notar a aplicação dessa solução de separação de quadrantes:



Questão 03

Considere o filtro passa baixa de Butterworth

$$H(u,v) = \frac{1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}$$

Calcule a expressão que representa o filtro passa alta de Butterworth, a partir desse filtro passa baixa. Dicas: Observe o que foi comentado no slide 57 da aula de filtragem. Lá, é para um filtro Gaussiano, mas o raciocínio é o mesmo. O resultado final a ser calculado está no slide 65; esse é o resultado a ser alcançado. Você deve calcular como chegar nele.

R.:

Sabendo que o filtro passa-alta nada mais é que o complementar do filtro passa baixa, temos então que:

$$H_{FPA}(u,v) = 1 - H_{FPB}(u,v)$$

Logo, substituindo a equação do filtro passa baixa de Butterworth, temos que:

$$H_{FPA}(u,v) = 1 - \frac{1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}} = \frac{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n} - 1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}} = \frac{\left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}$$

Então, sabendo que

$$\left[\frac{D(u,v)}{D_0}\right]^{2\cdot n} \cdot \left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n} = 1$$

logo:

$$H_{FPA}(u,v) = \frac{\left(\frac{D(u,v)}{D_0}\right)^{2\cdot n}}{\left(\frac{D(u,v)}{D_0}\right)^{2\cdot n}\left[1+\left(\frac{D(u,v)}{D_0}\right)^{-2\cdot n}\right]} = \frac{1}{1+\left(\frac{D(u,v)}{D_0}\right)^{-2\cdot n}}$$

Como

$$\left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n} = \left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}$$

Então temos:

$$H_{FPA}(u,v) = \frac{1}{1+\left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n}} = \frac{1}{1+\left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}}$$

$$\therefore H_{FPA}(u,v) = \frac{1}{1+\left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}} \square$$

Questão 04

A convolução discreta é uma operação comutativa. Ou seja:

f * h = h * f, onde * é a operação de convolução.

Comprove isso fazendo a convolução discreta dos dois filtros abaixo. Analise sua resposta.

(1 / 9) * f:

1 / 9	1 / 9	1 / 9
1 / 9	1 / 9	1 / 9
1 / 9	1 / 9	1 / 9

h:

-1	0	1
-1	0	1
-1	0	1

Ou seja, você deve calcular (e apresentar todos os cálculos) da convolução discreta das duas matrizes acima, operadas em ordens inversas: f * h e h * f.

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**.

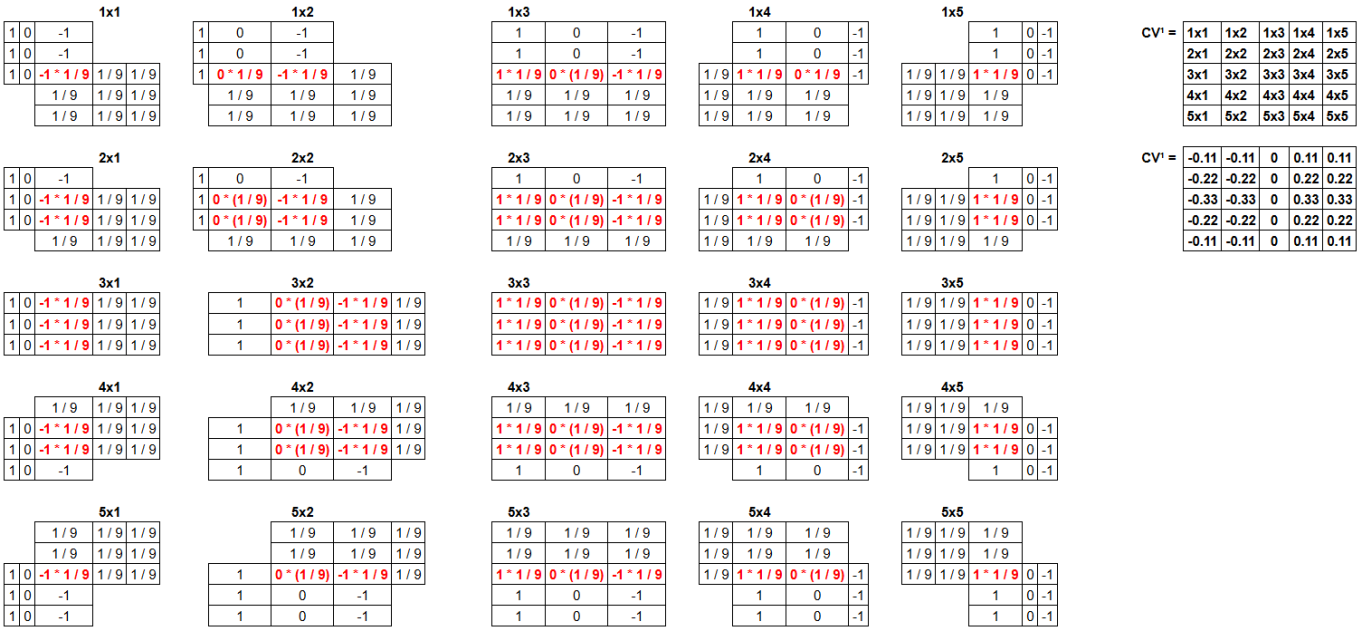
1. **Para f * h:**

Primeiramente, vamos rotacionar a segunda matriz, sendo ela **h**, ficando:

h:

1	0	-1
1	0	-1
1	0	-1

Então, aplicando a convolução teremos:



Logo, a matriz resultante da convolução será:

$$f * h = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix}$$

2. **Para h * f:**

Primeiramente, vamos rotacionar a segunda matriz, sendo ela **f**, devido a sua natureza, a rotação dela é igual a original, logo aplicando a revolução teremos:

1x1

1/9	1/9	1/9		
1/9	1/9	1/9		
1/9	1/9	-1 * 1/9	0	1
		-1	0	1
		-1	0	1

1x2

1/9	1/9	1/9	
1/9	1/9	1/9	
1/9	-1 * 1/9	0 * (1/9)	1
	-1	0	1
	-1	0	1

1x3

1/9	1/9	1/9
1/9	1/9	1/9
-1 * 1/9	0 * (1/9)	1 * 1/9
-1	0	1
-1	0	1

1x4

	1/9	1/9	1/9
	1/9	1/9	1/9
-1	0 * 1/9	1 * 1/9	1/9
-1	0	1	
-1	0	1	

1x5

		1/9	1/9	1/9
		1/9	1/9	1/9
-1	0	1 * 1/9	1/9	1/9
-1	0	1		
-1	0	1		

2x1

1/9	1/9	1/9		
1/9	1/9	-1 * 1/9	0	1
1/9	1/9	-1 * 1/9	0	1
		-1	0	1

2x2

1/9	1/9	1/9	
1/9	-1 * 1/9	0 * (1/9)	1
1/9	-1 * 1/9	0 * (1/9)	1
	-1	0	1

2x3

1/9	1/9	1/9
-1 * (1/9)	0 * (1/9)	1 * (1/9)
-1 * (1/9)	0 * (1/9)	1 * (1/9)
-1	0	1

2x4

	1/9	1/9	1/9
-1	0 * 1/9	1 * (1/9)	1/9
-1	0 * 1/9	1 * (1/9)	1/9
-1	0	1	

2x5

		1/9	1/9	1/9
-1	0	1 * (1/9)	1/9	1/9
-1	0	1 * (1/9)	1/9	1/9
-1	0	1		

3x1

1/9	1/9	-1 * 1/9	0	1
1/9	1/9	-1 * 1/9	0	1
1/9	1/9	-1 * 1/9	0	1

3x2

	1/9	-1 * (1/9)	0 * (1/9)	1
	1/9	-1 * (1/9)	0 * (1/9)	1
	1/9	-1 * (1/9)	0 * (1/9)	1

3x3

-1 * (1/9)	0 * (1/9)	1 * (1/9)
-1 * (1/9)	0 * (1/9)	1 * (1/9)
-1 * (1/9)	0 * (1/9)	1 * (1/9)

3x4

-1	0 * (1/9)	1 * (1/9)	1/9
-1	0 * (1/9)	1 * (1/9)	1/9
-1	0 * (1/9)	1 * (1/9)	1/9

3x5

-1	0	1 * (1/9)	1/9	1/9
-1	0	1 * (1/9)	1/9	1/9
-1	0	1 * (1/9)	1/9	1/9

4x1

		-1	0	1
1/9	1/9	-1 * (1/9)	0	1
1/9	1/9	-1 * (1/9)	0	1
1/9	1/9	1/9		

4x2

		-1	0	1
	1/9	-1 * (1/9)	0 * (1/9)	1
	1/9	-1 * (1/9)	0 * (1/9)	1
	1/9	1/9	1/9	

4x3

-1	0	1
-1 * (1/9)	0 * (1/9)	1 * (1/9)
-1 * (1/9)	0 * (1/9)	1 * (1/9)
1/9	1/9	1/9

4x4

-1	0	1	
-1	0 * (1/9)	1 * (1/9)	1/9
-1	0 * (1/9)	1 * (1/9)	1/9
	1/9	1/9	1/9

4x5

-1	0	1		
-1	0	1 * (1/9)	1/9	1/9
-1	0	1 * (1/9)	1/9	1/9
		1/9	1/9	1/9

5x1

		-1	0	1
		-1	0	1
1/9	1/9	-1 * (1/9)	0	1
1/9	1/9	1/9		
1/9	1/9	1/9		

5x2

		-1	0	1
		-1	0	1
	1/9	-1 * (1/9)	0 * (1/9)	1
	1/9	1/9	1/9	
	1/9	1/9	1/9	

5x3

-1	0	1
-1	0	1
-1 * (1/9)	0 * (1/9)	1 * (1/9)
1/9	1/9	1/9
1/9	1/9	1/9

5x4

-1	0	1	
-1	0	1	
-1	0 * (1/9)	1 * (1/9)	1/9
	1/9	1/9	1/9
	1/9	1/9	1/9

5x5

-1	0	1		
-1	0	1		
-1	0	1 * (1/9)	1/9	1/9
		1/9	1/9	1/9
		1/9	1/9	1/9

CV² =

1x1	1x2	1x3	1x4	1x5
2x1	2x2	2x3	2x4	2x5
3x1	3x2	3x3	3x4	3x5
4x1	4x2	4x3	4x4	4x5
5x1	5x2	5x3	5x4	5x5

CV² =

-0.11	-0.11	0	0.11	0.11
-0.22	-0.22	0	0.22	0.22
-0.33	-0.33	0	0.33	0.33
-0.22	-0.22	0	0.22	0.22
-0.11	-0.11	0	0.11	0.11

Analogo ao item anterior, temos:

$$h * f = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix}$$

3. **Conclusão:**

Logo, temos de fato que f * h = h * f.

$$f * h = h * f = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix} \square$$

Questão 05

Calcule uma janela para um possível filtro Gaussiano através da convolução discreta de três filtros Box, como abaixo:

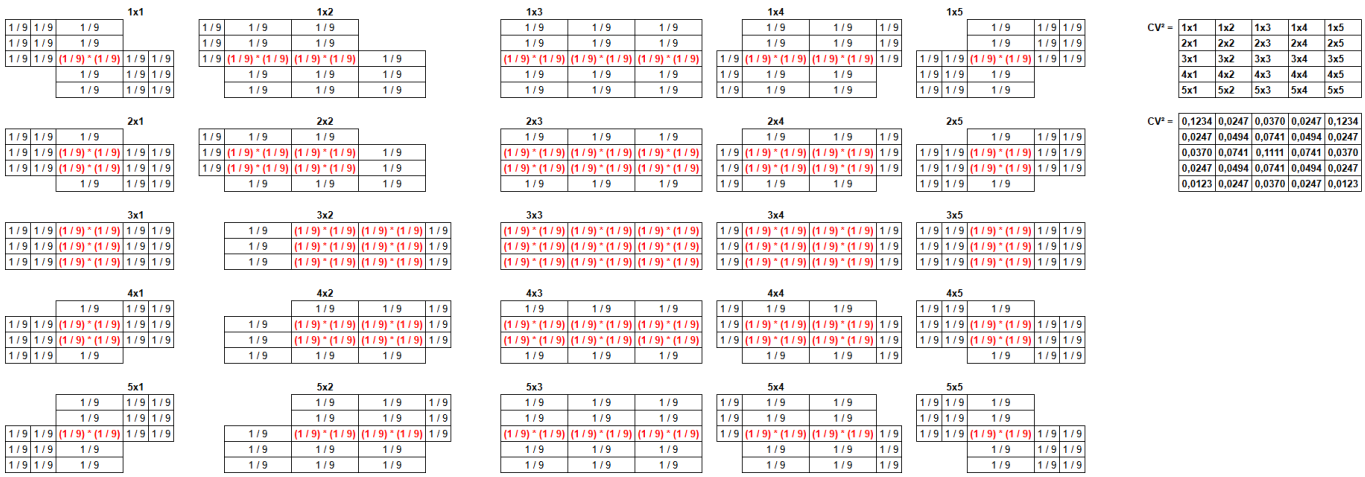
$$\text{Filtro Box} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**. Vamos tomar como **B = Filtro Box**, para simplificar.

1) B * B

Calculando, devido a simetria de B, a rotação será igual, logo temos:



Resultando em:

$$B * B = \begin{bmatrix} 0.0123 & 0.0247 & 0.0370 & 0.0247 & 0.0123 \\ 0.0247 & 0.0494 & 0.0741 & 0.0494 & 0.0247 \\ 0.0370 & 0.0741 & 0.1111 & 0.0741 & 0.0370 \\ 0.0247 & 0.0494 & 0.0741 & 0.0494 & 0.0247 \\ 0.0123 & 0.0247 & 0.0370 & 0.0247 & 0.0123 \end{bmatrix}$$

2) **(B * B) * B**

Analogo ao item anterior, devido a simetria de B, teremos rotação igual, logo temos:

Resultado em:

$$(B * B) * B = \begin{bmatrix} 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0096 & 0.0288 & 0.0576 & 0.0672 & 0.0576 & 0.0288 & 0.0096 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \end{bmatrix}$$

3. **Conclusão:**

Temos o produto resultante abaixo:

$$(B * B) * B = \begin{bmatrix} 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0096 & 0.0288 & 0.0576 & 0.0672 & 0.0576 & 0.0288 & 0.0096 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \end{bmatrix}$$

Questão 06

Considere a imagem abaixo, uma imagem em 16 tons de cinza. Apresente o resultado da aplicação de um filtro Box 3x3 (o mesmo Box da questão anterior) nessa imagem. Apresente os cálculos e explique todas as decisões tomadas para realizar a filtragem, observando que sua saída deve também ser uma imagem em 16 tons de cinza.

1	3	0
2	2	3
1	3	1

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**.

Como visto nas questões anteriores, a rotação da matriz de filtro box 3x3 é simétrica, então sua rotação será igual a ela mesma.

Como visto na aula de filtragem, apartir do slide 98, vamos aplicar uma convolução entre um filtro e uma imagem, logo usaremos a **Correlação Cruzada**, definindo como estratégia para a borda será a extensão nula, e apos aplicar o processo, multiplicaremos a constante 1 / 9.

Logo:

1x1

1	1	1	
1	1 * 1	1 * 3	0
1	1 * 2	1 * 2	3
	1	3	1

1x2

1	1	1
1 * 1	1 * 3	1 * 0
1 * 2	1 * 2	1 * 3
1	3	1

1x3

	1	1	1
1	1 * 3	1 * 0	1
2	1 * 2	1 * 3	1
1	3	1	

CV¹ =

1x1	1x2	1x3
2x1	2x2	2x3
3x1	3x2	3x3

CV¹ =

8	11	8
12	16	12
8	12	9

2x1

1	1 * 1	1 * 3	0
1	1 * 2	1 * 2	3
1	1 * 1	1 * 3	1

2x2

1 * 1	1 * 3	1 * 0
1 * 2	1 * 2	1 * 3
1 * 1	1 * 3	1 * 1

2x3

1	1 * 3	1 * 0	1
2	1 * 2	1 * 3	1
1	1 * 3	1 * 1	1

3x1

	1	3	0
1	1 * 2	1 * 2	3
1	1 * 1	1 * 3	1
1	1	1	

3x2

1	3	0
1 * 2	1 * 2	1 * 3
1 * 1	1 * 3	1 * 1
1	1	1

3x3

1	3	0	
2	1 * 2	1 * 3	1
1	1 * 3	1 * 1	1
	1	1	1

Então temos:

$$\frac{1}{9} \cdot (\text{Img} * \text{B}) = \frac{1}{9} \cdot \begin{bmatrix} 8 & 11 & 8 \\ 12 & 16 & 12 \\ 8 & 12 & 9 \end{bmatrix}$$

Por fim, como a saida deve ser uma imagem em 16 tons de cinza, temos que aplicar um processo de clip entre 0 e 15, ou seja, se xij < 0, o valor sera atribuido para 0; caso xij > 15, o valor será atribuido para 15. Logo temos:

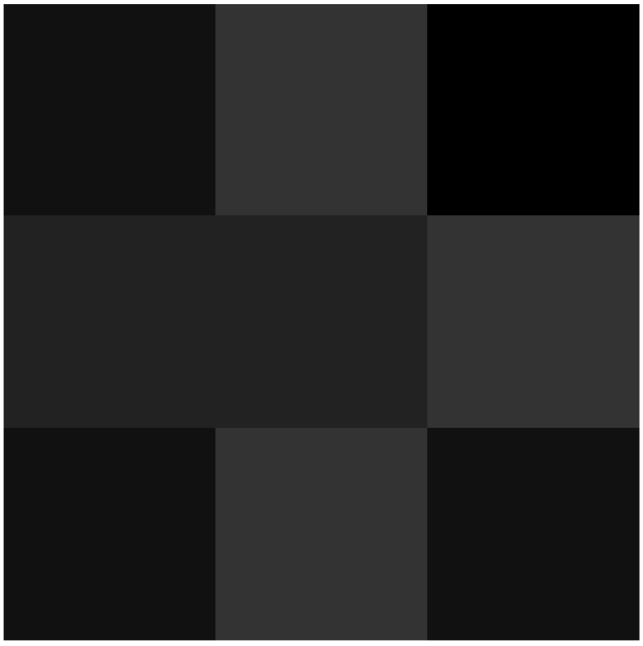
$$\frac{1}{9} \cdot (\text{Img} * \text{B}) = \frac{1}{9} \cdot \begin{bmatrix} 8 & 11 & 8 \\ 12 & 16 & 12 \\ 8 & 12 & 9 \end{bmatrix} = \frac{1}{9} \cdot \begin{bmatrix} 8 & 11 & 8 \\ 12 & 15 & 12 \\ 8 & 12 & 9 \end{bmatrix} = \begin{bmatrix} 0.8889 & 1.2222 & 0.8889 \\ 1.3333 & 1.6667 & 1.3333 \\ 0.8889 & 1.3333 & 1 \end{bmatrix}$$

Aplicando os devidos arredondamentos, temos:

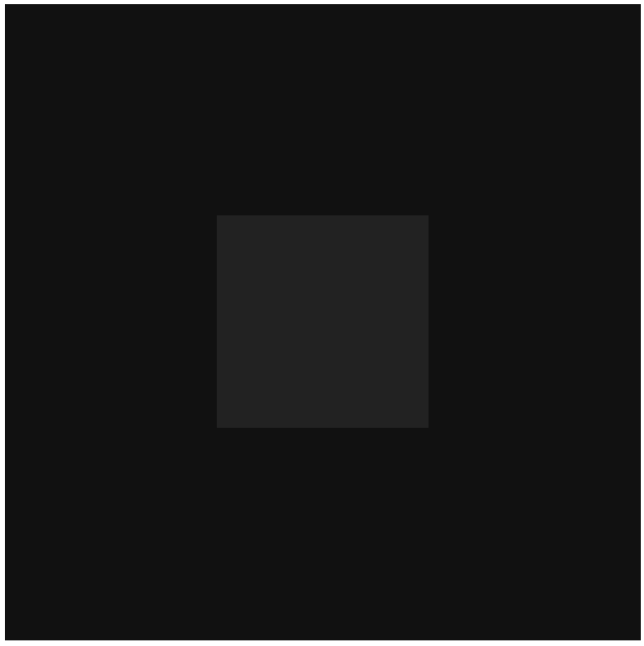
$$\frac{1}{9} \cdot (\text{Img} * \text{B}) = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

A ilustração gráfica do procedimento realizado esta abaixo:

Imagem Original



Com Filtro Box 3x3



Questão 07

Disserte sobre a seguinte afirmação:

"As operações morfológicas de Erosão e Dilatação, aplicadas com um mesmo elemento estruturante, não são, necessariamente, operações inversas uma da outra."

R.:

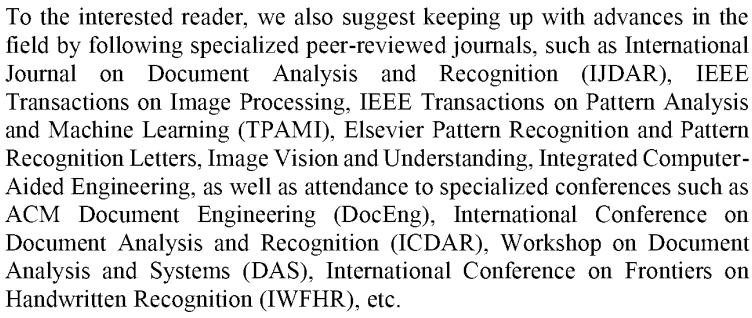
Enquanto a dilatação tem como objetivo expandir um objeto, a erosão visa torná-lo mais estreito. Essas operações não são necessariamente inversas, e isso pode ser exemplificado por um caso em que a erosão elimina um ponto isolado. Se uma dilatação for aplicada em seguida, esse ponto não poderá ser recuperado, uma vez que a dilatação expande apenas regiões já existentes. Se o ponto era isolado e não está mais presente na imagem, não há nada a ser expandido.

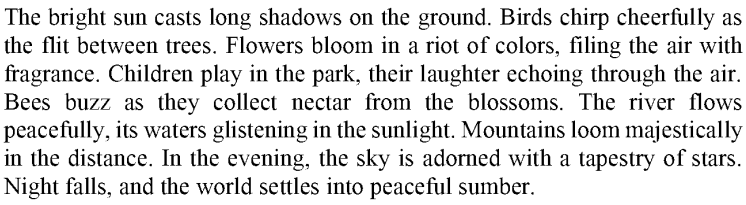
O cenário oposto também pode ocorrer: ao se aplicar uma dilatação, duas regiões próximas podem se unir. Após isso, uma erosão não será capaz de separá-las novamente, pois agora a imagem possui apenas uma única região contínua. Assim, não é possível aplicar uma suavização que recupere a separação original entre essas regiões.

Ao inves de serem tratadas como operações inversas, elas são complementares, os quais são definidos como Abertura (Suavisação de contornos em objetos, Remoção de ramos em objetos e Expansão de regiões de preto) e Fechamento (Preenchimento de falhas em regiões com contorno, diminuição de áreas de preto), a qual a primeira é a aplicação de uma erosão seguida de uma dilatação e a outra é uma dilatação seguida de uma erosão, ambas com o uso de um mesmo elemento estruturante.

Questão 08

Considere as imagens Book_1.png e Book_2.png disponibilizadas. Utilizando apenas técnicas de processamento de imagens, crie um algoritmo que verifique se essas imagens possuem a letra A ou não. Apenas o A maiúsculo deve ser procurado e não precisa retornar quantos têm; apenas se tem ou não. Observe que as imagens estão em preto e branco.


To the interested reader, we also suggest keeping up with advances in the field by following specialized peer-reviewed journals, such as International Journal on Document Analysis and Recognition (IJDAR), IEEE Transactions on Image Processing, IEEE Transactions on Pattern Analysis and Machine Learning (TPAMI), Elsevier Pattern Recognition and Pattern Recognition Letters, Image Vision and Understanding, Integrated Computer-Aided Engineering, as well as attendance to specialized conferences such as ACM Document Engineering (DocEng), International Conference on Document Analysis and Recognition (ICDAR), Workshop on Document Analysis and Systems (DAS), International Conference on Frontiers on Handwritten Recognition (IWFHR), etc.


The bright sun casts long shadows on the ground. Birds chirp cheerfully as the flit between trees. Flowers bloom in a riot of colors, filing the air with fragrance. Children play in the park, their laughter echoing through the air. Bees buzz as they collect nectar from the blossoms. The river flows peacefully, its waters glistening in the sunlight. Mountains loom majestically in the distance. In the evening, the sky is adorned with a tapestry of stars. Night falls, and the world settles into peaceful sumber.

R.:

Foi encontrado a seguinte solução para o problema, sendo ela:

1) **[Lógica] Via erosão**

Essa solução apresenta uma forma mais rápida para o match, aonde é recortado da propria matriz da imagem original o bloco referente a letra de interesse (`A`) e utilizado ela como `struct` na hora de aplicar a erosão, logo será buscado na imagem original locais a qual possua esse `struct`, em caso afirmativo sera colocado branco (ou pela logica da implementação, 1) para a região encontrada do algoritmo e preto (pela logica da implementação, 0) para a região de fora do `struct`.

1.1) **Lógica do Algoritmo**

1. **Recorte do Objeto de Interesse**
   Um recorte é realizado sobre a região de interesse da imagem, resultando em um template que será utilizado como struct.

2. **Binarização**
   Binarização das imagens, tanto original quanto template.

3. **Aplicação da Erosão**
   A operação de erosão é aplicada na imagem binarizada utilizando o template binarizado como struct.

4. **Verificação da Presença da Letra "A"**
   Após a erosão, verifica-se se a matriz resultante contém ao menos um pixel com valor 1 (branco). Isso é feito somando os valores da matriz de erosão.

5. **Conclusão**

$$\text{Resultado} = \begin{cases} \text{Letra 'A' encontrada,} & \text{se } \sum_{i=1}^{n} \sum_{j}^{m} E_{ij} > 0 \\ \text{Letra 'A' não encontrada,} & \text{se } \sum_{i=1}^{n} \sum_{j}^{m} E_{ij} = 0 \end{cases}$$

Sendo:

$$\{ E, \quad \text{matriz de Erosão n x m}$$

1.2) **Vantagens**

Método rápido e facil de aplicar.

1.3) **Problema**

Necessita que o corte para a imagem de template seja o mais preciso possivel, também é ainda mais sensivel a mudança de pixels e troca de fontes. Ao invés de recortar diretamente da imagem, e sim tirar um print da letra de interesse, o método pode não funcionar, uma vez que ele busca um struct especifico de pixels alinhados.

2) **[Implementação] Via erosão**

2.0) **Leitura de imagens**

```
image_path_1 = str(path_assets / "atv02_lista02-assets" / "Book_1.png")
image_path_2 = str(path_assets / "atv02_lista02-assets" / "Book_2.png")

image_book1 = cv2.imread(image_path_1, cv2.IMREAD_GRAYSCALE)
image_book2 = cv2.imread(image_path_2, cv2.IMREAD_GRAYSCALE)
```

2.1) **Definição de cortes**

Nesse momento, é feito a seleção de alguns cortes na imagem, para a seleção da letra de interesse para uso de template e `struct`. Cortes 2 e 3 são apenas ilustrativos do processo.

```
# Configurando cortes das imagens para procura
cortes = {
    "Imagem completa": ((0, image_book1.shape[0]), (0,
image_book1.shape[1])),
    "Primeiro corte": ((0, image_book1.shape[0]), (55, 87)),
    "Segundo corte": ((470, 570), (55, 87)),
    "Template final": ((501, 529), (57, 86))
}
```

Agora visualizando os cortes, temos:

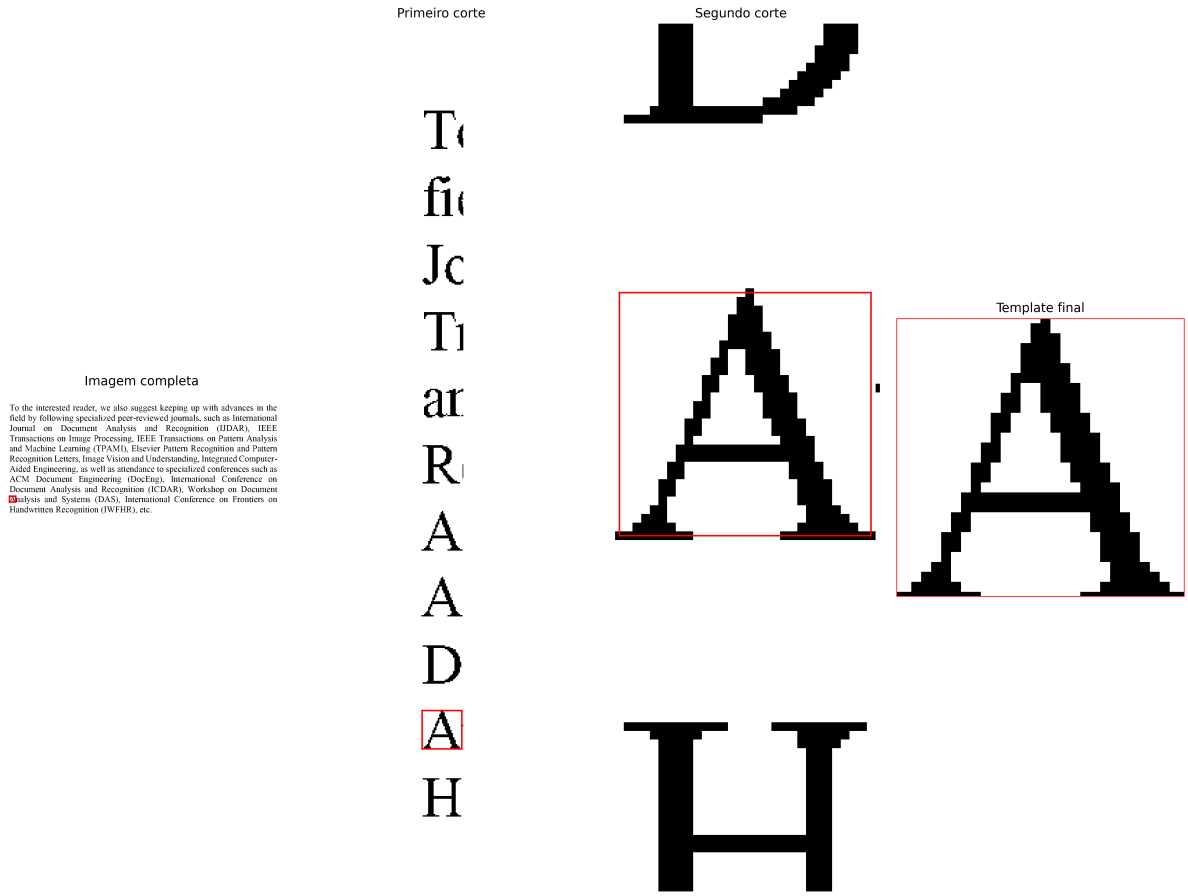
```
fig, ax = plt.subplots(1, len(cortes), figsize=(16, 12))
if not isinstance(ax, np.ndarray):
    ax = np.array([ax])
for idx, (title, sliced) in enumerate(cortes.items()):
    ax[idx].imshow(image_book1, cmap='gray')
    ax[idx].axis('off')
    if idx > 0:
        ax[idx].set_ylim(sliced[0][1], sliced[0][0])
        ax[idx].set_xlim(sliced[1][0], sliced[1][1])

    #if idx == 0:
    x_init, x_end = cortes["Template final"][1]
    y_init, y_end = cortes["Template final"][0]
    ax[idx].plot(
        [x_init] * np.ones(len(np.arange(y_init, y_end +1 ))),
        np.arange(y_init, y_end + 1),
        color='red',
        label='Média da linha'
    )

    ax[idx].plot(
        [x_end] * np.ones(len(np.arange(y_init, y_end+ 1))),
        np.arange(y_init, y_end + 1),
        color='red',
        label='Média da linha'
    )
    ax[idx].plot(
        np.arange(x_init, x_end+ 1),
        [y_init] * len(np.arange(x_init, x_end+ 1)),
        color='red',
        label='Média da linha'
    )

    ax[idx].plot(
        np.arange(x_init, x_end+ 1),
        [y_end] * len(np.arange(x_init, x_end+ 1)),
        color='red',
        label='Média da linha'
    )
    ax[idx].set_title(title)

fig.tight_layout()
fig.savefig(path_assets / "atv02-q08-s1-00.png", dpi=400,
bbox_inches='tight')
plt.show()
```

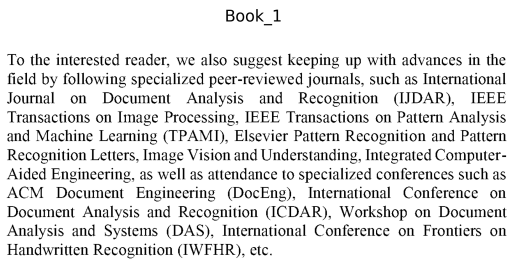


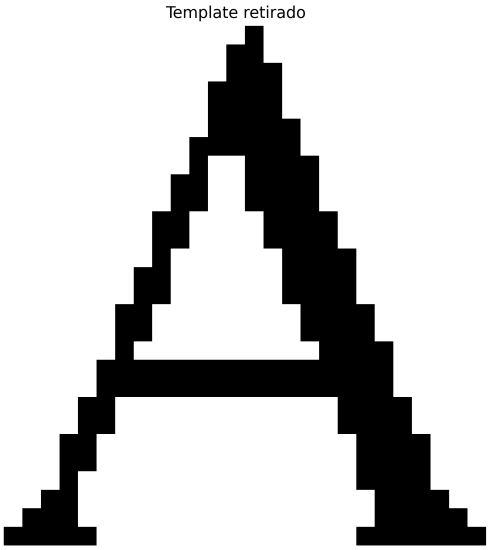
Template utilizado então será:

```
template = image_book1[
    slice(*cortes["Template final"][0]),
    slice(*cortes["Template final"][1])
]
fig, ax = plt.subplots(1, 2, figsize=(16, 10))

ax[0].imshow(
    image_book1, cmap="gray"
)
ax[0].set_title("Book_1")
ax[1].imshow(
    template,
    cmap='gray'
)
ax[1].set_title("Template retirado")
for _ax in ax:
    _ax.axis("off")

fig.savefig(path_assets / "atv02-q08-s1-01.png", dpi=400,
bbox_inches='tight')
```





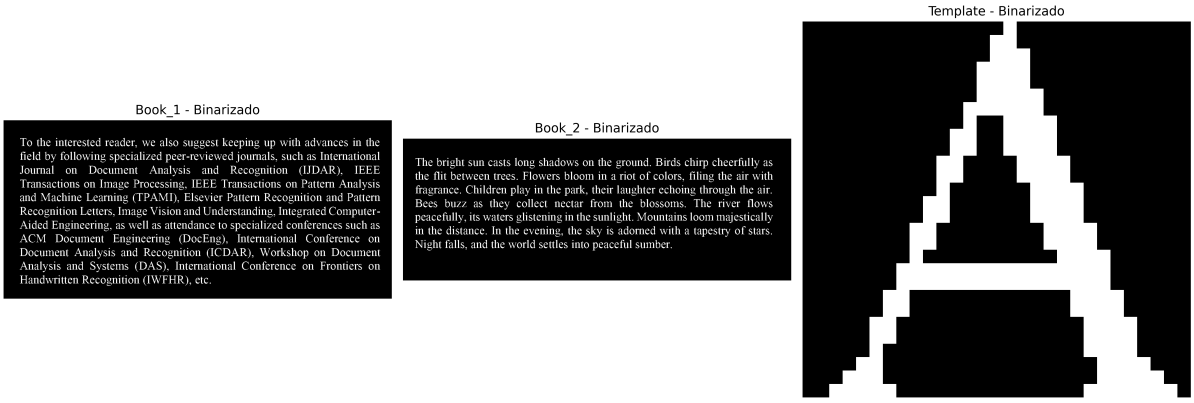
2.2) **Binarização**

Aqui será feita a binarização da image, para fundo preto e letra branca.

```
# binarização
_, template_mask = cv2.threshold(template, 127, 255, cv2.THRESH_BINARY_INV)
_, image_mask_1 = cv2.threshold(image_book1, 127, 255,
cv2.THRESH_BINARY_INV)
_, image_mask_2 = cv2.threshold(image_book2, 127, 255,
cv2.THRESH_BINARY_INV)

# dicionario a ser usado posteriormente
imgs_mask = {
    "Book_1 - Binarizado": image_mask_1,
    "Book_2 - Binarizado": image_mask_2,
    "Template - Binarizado": template_mask
}
lista_books = list(imgs_mask.keys())[:(-1)] # seleção apenas dos books
(exceto o template)

# Plot
fig, ax = plt.subplots(1, len(imgs_mask.keys()), figsize=(16, 10))
for (title, content), _ax in zip(imgs_mask.items(), ax.flatten()):
    _ax.imshow(
        content,
        cmap="gray"
    )
    _ax.set_title(title)
    _ax.axis("off")
fig.tight_layout()
fig.savefig(path_assets / "atv02-q08-s1-02.png", dpi=400,
bbox_inches='tight')
plt.show()
```



2.3) **Erosão**

Neste momento, vamos aplicar o processo de erosão sobre o template, que será considerado o nosso struct.

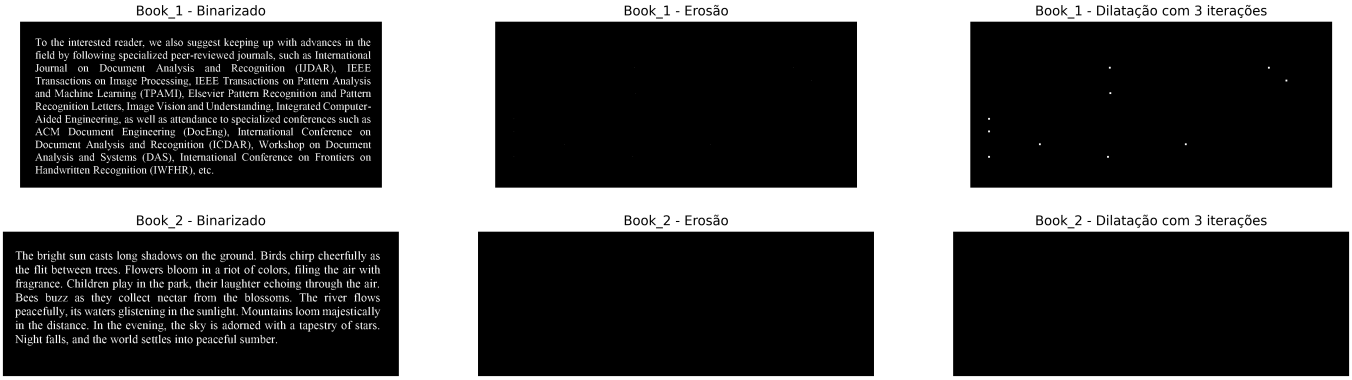
```
# aplicar erosão morfológica usando o template como struct
eroded_results = {}
for name in lista_books:
    eroded_results[name] = {
        "eroded": cv2.erode(
            imgs_mask[name],
            template_mask,
            iterations=1
        ) / 255  # Normalizando para [0, 1]
    }
    eroded_results[name]["contains_A"] = bool(eroded_results[name]
["eroded"].sum() > 0)
    eroded_results[name]["total-of_A"] = int(eroded_results[name]
["eroded"].sum())
```

Para fins apenas ilustrativos, vamos dilatar o resultado dessa erosão apenas para conseguir plotar os pontos que foram encontrados de com o `struct`.

```
# Aplicando uma dilatação apenas para visualizar melhor a localização dos A
encontrados
dilated = {}
k_iter = 3
for name in lista_books:
    dilated[name] = cv2.dilate(
        eroded_results[name]["eroded"].astype(np.uint8),
        kernel=np.ones((3, 3), np.uint8), iterations=k_iter
    )

fig, ax = plt.subplots(2, 3, figsize=(22, 6))
for idx, lista in enumerate(lista_books):
    ax[idx, 0].imshow(
        imgs_mask[lista],
        cmap="gray"
```

```
    )
    ax[idx, 1].imshow(
        eroded_results[lista]["eroded"],
        cmap="gray"
    )
    ax[idx, 2].imshow(
        dilated[lista],
        cmap="gray"
    )
    for _ax in ax[idx, :]:
        _ax.axis("off")
    ax[idx, 0].set_title(lista)
    ax[idx, 1].set_title(lista.split(" - ")[0] + " - Erosão")
    ax[idx, 2].set_title(lista.split(" - ")[0] + f" - Dilatação com
{k_iter} iterações")
fig.savefig(path_assets / "atv02-q08-s1-03.png", dpi=400,
bbox_inches='tight')
```



Com a erosão seguida com a dilatação, considerando 3 iterações, é mais facil de visualizar graficamente a presença da letra "A".

2.4) **Conclusão**

```
for key, content in eroded_results.items():
    print("-" * 30)
    print(key.split(" - ")[0])
    print(f"Possuí a letra A: {content['contains_A']}")
    if content["contains_A"]:
        print(f"Total de letra A: {content['total-of_A']}")
print("-"*30)
```

```
------------------------------
Book_1
Possuí a letra A: True
Total de letra A: 10
------------------------------
Book_2
```

```
Possuí a letra A: False
------------------------------
```

Apos a aplicação dessa solução, temos:

1. `Book_1`: a letra `A` foi detectada na imagem, além de terem sido identificados 10 letras na imagem, pela logica do algoritmo implementado no decorrer da solução.

2. `Book_2`: a letra `A` não foi detectada na imagem, pela logica do algoritmo implementado no decorrer da solução.

Questão 09

Considere a imagem `cameraman_pattern.png`. Tente eliminar o padrão de linhas que aparece nela, da melhor forma possível, usando:

a) uma solução aplicada no domínio da frequência;

b) uma solução aplicada no domínio espacial.

R.:

- **Obs.:** o código em notebook para essa questão pode ser encontrado clicando no link.

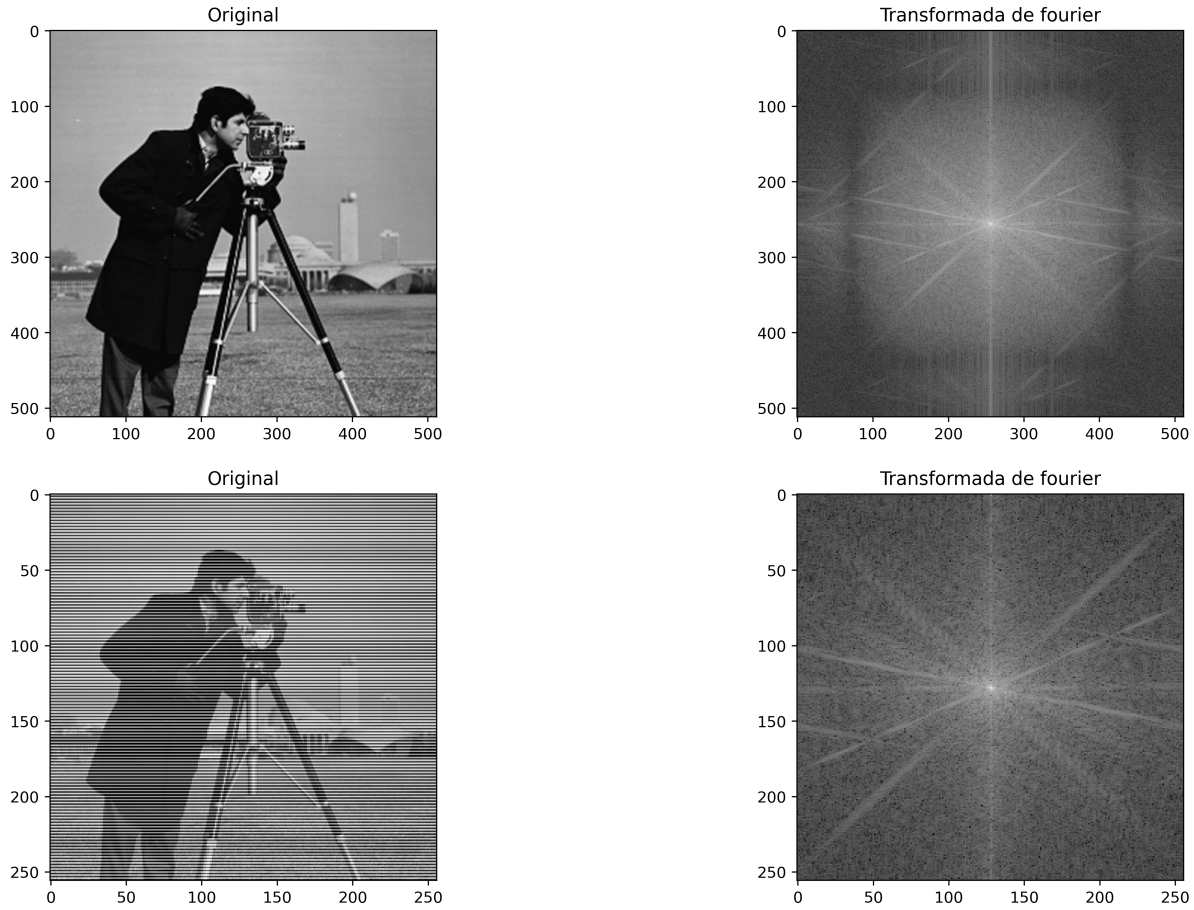
Lendo as imagens

```
img_names = ["cameraman.png", "cameraman_pattern.png"]
# img = cv2.imread(path_assets / "atv02_lista02-assets" /
'cameraman_pattern.png', cv2.IMREAD_GRAYSCALE)
imgs = {
    i.split(".")[0]: cv2.imread(path_imgs_atv / i, cv2.IMREAD_GRAYSCALE)
for i in img_names
}
```

Visualizando

```
fourier = {
    name: {
        "tft": apply_fourier_transform(img)[1],
        "spectrum": apply_fourier_transform(img)[0]
    } for name, img in imgs.items()
}
# Mostrar imagem resultante
fig, ax = plt.subplots(len(fourier), 2, figsize=(16, 10))
for _ax, name in zip(ax, fourier.keys()):
    _ax[0].imshow(imgs[name], cmap='gray')
    _ax[0].set_title('Original')
    _ax[1].imshow(fourier[name]["spectrum"].astype(float), cmap='gray')
```

```
    _ax[1].set_title('Transformada de fourier')
    #_ax[0].axis('off')
    #_ax[1].axis('off')
fig.savefig(path_assets / "atv02-q09-init.png", bbox_inches='tight',
dpi=400)
plt.show()
```



a)

Usaremos um filtro passa baixa gaussiano, aplicando um range de valores de σ para apenas selecionar o melhor valor para a imagem em questão e poder ilustrar o impacto da alteração desse valor ao longo sobre a imagem. O range de busca será σ = {0.3, 0.5, 0.7, 0.9, 1, 1.5, 3}

```
img = imgs["cameraman_pattern"]

# Aplicar a Transformada de Fourier
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

# Definir os ranges e parametros
nx = dft.shape[1]
ny = dft.shape[0]
cxrange = np.concatenate((np.arange(0, nx // 2 + 1), np.arange(-nx // 2 +
1, 0)))
cyrange = np.concatenate((np.arange(0, ny // 2 + 1), np.arange(-ny // 2 +
```

```
1, 0)))
cx, cy = np.meshgrid(cxrange, cyrange)
fxrange = cxrange * 2 * np.pi / nx
fyrange = cyrange * 2 * np.pi / ny
fx, fy = np.meshgrid(fxrange, fyrange)

# Lista de sigmas a serem procurados
sigmas = [0.3, 0.5, 0.7, 0.9, 1, 1.5, 3]

fig, ax = plt.subplots(2, (len(sigmas) // 2) + 1, figsize=(16, 10))
ax = ax.flatten()
ax[0].imshow(img, cmap='gray')
ax[0].set_title('Original')
for _ax in ax:
    _ax.axis('off')
for idx, sigma in enumerate(sigmas):
    # Filtro passa-baixa gaussiano
    ms = np.exp(-(fx**2 + fy**2) / (2 * (sigma**2)))

    # Aplicar o filtro
    smoothF = dft * ms
    smooth = np.abs(ifft2(smoothF))

    # Exibir a imagem suavizada
    ax[idx + 1].imshow(smooth.round().astype(int), cmap='gray')
    ax[idx + 1].set_title("Filtro Passa-Baixa Gaussiano " + rf"($\sigma=
{sigma}$)")
    ax[idx + 1].axis('off')
fig.savefig(path_assets / "atv02-q09-a1.png", bbox_inches='tight', dpi=400)
plt.show()
```

Conclusão

Analisando detalhadamente, podemos notar que os filtros considerando σ = 3 e σ = 0.3 podem ser descartados, uma vez que um apresentou ainda a presença do tracejado horizontal na imagem, enquanto o outro apresenta um grau de borramento maior, quase sendo imperceptivel detectar detalhes do fundo da imagem, respectivamente.

Considerando o σ = 1.5, a imagem mantem um nivel de detalhes consideravel, contudo, ainda que leve, a imagem ainda apresenta linhas tracejadas no fundo, algumas sendo bem evidentes. Os valores de σ = {0.5, 0.7, 0.9} apresentaram um pouco mais de detalhes, mas um nivel de borramento ainda elevado.

Por fim, o σ ideal para esse problema pode ser considerado 1, uma vez que ele conseguiu conservar detalhes da imagem, desparecendo com as linhas horizontais e ainda apresentando um nível de borramento mínimo.

b)

Baseado na lógica do filtro box 3x3 apresentado em aula, vamos alterar um pouco a matriz h de tal forma que encontre a melhor máscara a ser aplicada. Dessa forma, vamos aplicar um conjunto de máscaras sobre a imagem, aplicando uma correlação cruzada. Abaixo temos as mascaras a serem testadas, foram selecionadas apartir de um conjunto de outras mascaras similares, e selecionadas apenas essas samples, com correções efetivas que elas causaram (Mascaras 01 e 02) e as que não conseguiram realizar a tarefa proposta (Mascaras 03 e 04)

$$\text{Máscara 01} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad \text{Máscara 02} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\text{Máscara 03} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{Máscara 04} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

```
## metodo usando filtro linear
# mascaras utilizadas
hs = {
    "Mascara 01": np.array(
        [
            [0, 0, 0],
            [0, 1, 1],
            [0, 1, 1]
        ]
    ),
    "Mascara 02": np.array(
        [
            [0, 1, 0],
            [1, 0, 1],
            [0, 1, 0]
        ]
    ),
    "Mascara 03": np.array(
        [
            [1, 1, 1],
```

```
            [1, 0, 1],
            [1, 1, 1]
        ]
    ),
    "Mascara 04": np.array(
        [
            [1, 0, 1],
            [0, 0, 0],
            [1, 0, 1]
        ]
    ),
}

for key, h in hs.items():
    # Normaliza a máscara
    h = h / np.sum(h)

# aplicação da correlação cruzada
filtered_img = {
    key: convolve2d(
        imgs["cameraman_pattern"],
        h,
        mode='same',
        boundary='fill',
        fillvalue=0
    ).round().astype(int) for key, h in hs.items()
}

fig, ax = plt.subplots(2, len(filtered_img.keys()) + 1, figsize=(18, 8))

ax[0, 0].set_title('Original')
ax[0, 0].imshow(imgs["cameraman_pattern"], cmap='gray')
ax[0, 0].axis('off')

ax[1, 0].set_title('Original (sem linhas)')
ax[1, 0].imshow(imgs["cameraman"], cmap='gray')
ax[1, 0].axis('off')

# Exibir as máscaras e as imagens filtradas
for _ax, (title, content) in zip(ax.T[1:], filtered_img.items()):
    # Normalizar a máscara para  [0, 1]
    mask = hs[title]
    mask_normalized = (mask - mask.min()) / (mask.max() - mask.min())

    # _ax[0].imshow(mask_normalized, cmap='gray')
    sns.heatmap(
        mask_normalized,
        ax=_ax[0],
        cbar=False,
        annot=True,
        fmt=".0f",
        annot_kws={"color": "black", "fontsize": 12, "weight": "bold"},
        linewidths=0.5,
        linecolor='black',
```

```
            cmap='binary',
            vmin=1, vmax=1
        )
        _ax[0].set_title(title)
        _ax[0].axis('off')

        _ax[1].imshow(content, cmap='gray')
        _ax[1].set_title(f"Filtrado ({title})")
        _ax[1].axis('off')

    fig.tight_layout()
    fig.savefig(path_assets / "atv02-q09-b2.png", dpi=400, bbox_inches='tight')
    plt.show()
```

Conclusão

Entre os filtros testadas, o Filtro 1 apresentou os melhores resultados, conseguindo remover completamente as linhas horizontais presentes na imagem cameraman_pattern.png. Além disso, essa máscara conseguiu manter os detalhes da imagem original e apresentou o menor nível de borramento.

O filtro 2 também foi capaz de eliminar as linhas horizontais, mas apresentou um borramento mais evidente, o que comprometeu visualmente a qualidade da imagem. Por outro lado, os filtros 3 e 4 não foram eficazes. Ambas não foram eficientes em remover as linhas horizontais e ainda introduziram um efeito de borramento significativo, prejudicando a nitidez e os detalhes da imagem.

Dessa forma, temos que o **filtro 1** foi a mais adequada para resolver o problema, equilibrando a remoção do padrão de linhas com a preservação da qualidade da imagem.

- **Filtro selecionado:**

$$h = \left(\frac{1}{4}\right) \cdot \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Questão 10

Análise Escala-Espaço: Observamos o mundo ao nosso redor em diferentes escalas. Sempre temos, à nossa frente, objetos mais próximos e objetos mais distantes. Nesse sentido, tomando como base a teoria de Marr, Pietro Perona e Jitendra Malik propuseram um método de detecção de bordas, usando a teoria escala-espaço. A ideia principal é que, em diferentes escalas da imagem, diferentes bordas se sobressaem. Essa variação de escala poderia ser conseguida com a aplicação de sucessivos resizes na imagem. No entanto, como cada resize tem como consequência, além da mudança de dimensões de uma imagem, a perda de detalhes, essa operação foi substituída pela aplicação de filtros Gaussianos de diferentes parâmetros. À medida que o filtro se torna mais forte (maior o desvio padrão), mais embaçada a imagem fica o que gera a perda de detalhes esperada.

Comprove a aplicação da teoria escala-espaço na detecção de bordas com o seguinte experimento: na imagem cameraman.png, aplique três filtros gaussianos diferentes (parte de um mais fraco e vá até um mais forte), detecte as bordas de cada um com o detector de Canny (precisa usar os mesmos parâmetros para todas as imagens) e, por fim, avalie os resultados finais, verificando se diferentes bordas foram encontradas.

OBS: Não se preocupe em achar um resultado final de boa qualidade; não é esse o objetivo do experimento

R.:

- **Obs.:** o código em notebook para essa questão pode ser encontrado clicando no link.

O intuito desse experimento é verificar a detecção das bordas apos a aplicação de três filtros gaussianos, e analisar o contorno encontrado apos cada filtro. Dessa forma, o esperado é que o nível de detalhe caia, a medida que se aumenta o sigma do filtro gaussiano, causando um efeito de perca de detalhes.

- **Leitura**

```
img_names = ["cameraman.png"]
imgs = {
    i.split(".")[0]: cv2.imread(path_imgs_atv / i, cv2.IMREAD_GRAYSCALE)
for i in img_names
}
```

- **Configurando parâmetros**

```
# Sigmas para o filtro gaussiano
sigmas = [1, 2, 4]

# Parametros para o método de deteccao de canny
low = 50
high = 150
```

- **Implementação e geração de gráficos**

```
fig, ax = plt.subplots(1, len(sigmas), figsize=(16, 8))
ax = ax.flatten()
for i, sigma in enumerate(sigmas):
    # Aplica filtro Gaussiano com sigma definido
    blurred = cv2.GaussianBlur(
        imgs["cameraman"],
        (0, 0),
        sigmaX=sigma,
        sigmaY=sigma
    )

    # Detecta bordas com Canny
    edges = cv2.Canny(
        blurred,
        low,
        high
    )

    # Exibe os resultados
    ax[i].imshow(edges, cmap='gray')
    ax[i].set_title(f'Canny{low, high} | Filtro gaussiano:' + rf'$\sigma$=
{sigma}')
    ax[i].axis('off')
fig.tight_layout()
fig.savefig(path_assets / "atv02-q10-02.png", dpi=400, bbox_inches='tight')
plt.show()
```

- **σ = 1**: existe diversas bordas, mostrando um nivel de detalhe bem significante, preservando pontos princiapis da imagem e secundários também. Isso se da possivelmente pelo valor de σ baixo, causando um efeito de emabaçamento menor.

- **σ = 2**: destaque nos detalhes principais, sumindo parcialmente traços mais secundários como os da grama. O filtro Gaussiano mais forte suaviza regiões pequenas, e o detector de bordas começa a destacar contornos mais marcantes.

- **σ = 4**: apenas as bordas principais da imagem são visíveis. Com o aumento da suavização, diversos detalhes desaparecem, restando apenas as transições de contraste mais evidentes.

Essa progressão demonstra que, ao aumentar o valor de σ no filtro Gaussiano, esta focando na imagem apenas os pontos mais caracteristicos, ou seja, apenas bordas maiores e mais importantes são conservadas. Dessa forma, diferentes bordas aparecem em diferentes escalas, demonstrando a ideia da análise escala-espaço como uma ferramenta bem interessante na segmentação de imagens com muitos objetos visuais.